

Tema 8 - Programación Orientada a Objetos en Python

Francisco Gortázar

POO y Python

- Python permite definir clases y crear objetos a partir de las mismas
- La forma de trabajar con clases y objetos es diferente a lenguajes con tipado estático
- Veremos estas diferencias y cómo se abordan en Python

Clases

- Una clase es una plantilla a partir de la cual puedo crear objetos
- En Python las clases definen su propio espacio de nombres
- Ejemplo:

```
class Fraction:
    def realValue(...) {
    }
```

Clases

- Los métodos de una clase siempre reciben como primer argumento **self**
 - **self** es una referencia al propio objeto que recibe el mensaje
 - equivalente al **this** de Java
 - es obligatorio ponerlo como primer parámetro
 - lo inyecta automáticamente el intérprete (no hay que pasarlo al método)

```
class Fraction:
    def realValue(self) {
    }
# f es un objeto Fraction:
f.realValue()
```

Clases

- Los atributos no se declaran
- Para crearlos se inicializan en el constructor
- El constructor es un método especial `__init__()`
 - Es llamado automáticamente al crear un objeto
 - Recibe `self` como primer parámetro
 - Los atributos deben cualificarse con `self` (si no serían variables locales al método)

```
class Fraction:
    def __init__(self, numerator, denominator) {
        self.num = numerator
        self.den = denominator
    }
```

Clases

- Los métodos acceden a los atributos usando `self`
- Así se distinguen de variables locales

```
class Fraction:
    def __init__(self, numerator, denominator):
        self.num = numerator
        self.den = denominator

    def realValue(self):
        return self.num / self.den
```

Objetos

- Para crear un objeto a partir de una clase usamos el nombre de la misma y pasamos los parámetros que requiera el constructor
 - Por debajo se llama al método `__init__()`
 - **NUNCA** pasamos el parámetro `self`
- Ejemplo:

```
f = Fraction(2,3)
print(f.realValue())
```

Más sobre atributos

- Hemos visto que los atributos no se declaran
 - No existen hasta que se inicializan
 - Si intentamos acceder a un atributo no inicializado obtenemos un error
 - El mismo que al acceder a un método que no existe

Más sobre atributos

- Como los atributos no se declaran:
 - No están disponibles hasta que se inicializan
 - No se pueden hacer privados (son públicos por defecto)
 - Pueden ser accedidos directamente usando el nombre del objeto
 - Pueden ser modificados desde fuera (no hay encapsulación)

```
from ejem1 import Fraccion

f = Fraccion(2,3)
print(f.realValue()) # 0.6666666666666666

f.num = 1
print(f.realValue()) # 0.3333333333333333
```

Más sobre atributos

- ¿Cómo solucionarlo?
 - Por convención: para indicar que un atributo es privado y no debería tocarse, su nombre empieza por “_” (guión bajo)
 - El que usa la clase debe respetar esta convención o arriesgarse a que la clase no funcione como espera

```
class Fraction:
    def __init__(self, numerator, denominator):
        self._num = numerator
        self._den = denominator

    def realValue(self):
        return self._num / self._den
```

Herencia

- Podemos crear jerarquías de herencia entre clases:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def comer(self):
        print("Estoy comiendo")

class Dog(Animal):
    def comunicar(self):
        print(f"Guau! Soy {self.name}")

class Cat(Animal):
    def comunicar(self):
        print(f"Miau! Soy {self.name}")
```

Herencia

- No necesito definir los métodos que quiera que redefinan las clases hijas en la clase padre
 - No hay necesidad -> no se comprueban tipos
 - Si el método está se invoca
 - Si no, se obtiene un error

```
d = Dog("Vega")
c = Cat("Minino")

d.comer()
c.comer()

d.comunicar()
c.comunicar()

d.dormir()
```

Herencia

- Todas las llamadas son polimórficas por defecto:
 - Si un método no se encuentra en la clase hija, se busca en la clase padre
 - Así sucesivamente hasta la raíz de la jerarquía
 - Si sobrescribo un método en la clase hija, se utilizará ese método siempre que el objeto sea de esa clase hija

Herencia

- Si tenemos una relación de herencia y la clase padre define un constructor que queremos llamar desde el de la clase hija usamos `super()`

```
class Animal:
    def __init__(self, name):
        self._name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self._breed = breed
```